



PymooLab

Manual do Usuário & Referência Técnica

Framework Visual de Análise para Otimização Multi-Objetivo

METISBr

Grupo de Pesquisa Brasileiro em Otimização
Multi-Objetivo e Many-Objective (MaOPs)

UFOP — Brasil

Versão: 2026.03 — Gerado em 03/03/2026

Contato: santostf+metisbr@ufop.edu.br

Sumário

1. Introdução
 2. Requisitos e Instalação
 3. Arquitetura do Projeto
 4. Estrutura de Diretórios
 5. Módulos Principais (Core)
 6. Sistema de Plugins
 7. Catálogo Implementado
 8. Criando Novos Problemas
 9. Criando Novos Algoritmos
 10. Criando Novas Métricas
 11. Criando Novos Operadores
 12. Execução e Uso do PymooLab
- Apêndice: Referências

1. Introdução

PymooLab é um framework open-source de análise visual para otimização multi-objetivo, construído sobre o ecossistema **pymoo** e a interface gráfica **PySide6**. O framework integra:

- Orquestração de benchmarks experimentais com execução paralela
- Descoberta automática de plugins locais (algoritmos, problemas, métricas, operadores)
- Sistema de análise estatística (Friedman, Wilcoxon) com exportação LaTeX/CSV
- Visualização interativa de fronteiras de Pareto e curvas de convergência
- Suporte a aceleração JAX/GPU para problemas e operadores
- Geração assistida por LLM (Anthropic) de artefatos de otimização
- Tomada de decisão multi-critério (MCDM) para seleção de soluções compromisso
- Reprodutibilidade garantida via planos de seed e manifestos de execução com SHA-256

O PymooLab foi desenvolvido pelo grupo **METISBr** na UFOP (Universidade Federal de Ouro Preto), com foco em pesquisa acadêmica em Otimização Multi-Objetivo e Many-Objective (MaOPs).

2. Requisitos e Instalação

2.1. Ambiente Recomendado

Item	Valor
Python	3.11.x
Backend padrão	CPU (NumPy)
Backend opcional	JAX (GPU/TPU)
GUI	PySide6 + qt-material

2.2. Dependências Base

Pacote	Versão
numpy	>=2.3, <3
pymoo	>=0.6.1, <0.7
scipy	>=1.16, <2
PySide6	>=6.10, <7
qt-material	>=2.17, <3
qt-material-icons	>=0.4, <1
matplotlib	>=3.10, <4
psutil	>=7.2, <8

2.3. Perfis Opcionais

Perfil	Pacotes	Uso
GPU/JAX	jax, jaxlib	Aceleração em GPU/TPU
LLM	anthropic	Geração de código assistida por IA
ML	scikit-learn	Algoritmos dependentes de ML
Dev	pytest	Testes automatizados

2.4. Instalação

```
pip install numpy pymoo scipy PySide6 qt-material qt-material-icons matplotlib psutil
pip install jax jaxlib                # opcional: GPU/JAX
pip install anthropic                  # opcional: LLM
pip install scikit-learn                # opcional: algoritmos ML
pip install pytest                      # opcional: desenvolvimento
```

3. Arquitetura do Projeto

O PymooLab segue uma arquitetura modular em camadas:

Camada	Módulo(s)	Responsabilidade
GUI	PymooLab.py, styles.py	Interface gráfica PySide6, estilização
Core Logic	pymoolab_core/	Análise estatística, execução, LLM, MCDM, registro
Base Classes	core/	Wrappers de compatibilidade do pymoo
Plugins	algorithms/, problems/, metrics/, operators/	Implementações auto-descobertas
Utilities	util/	Array backend, display, NDS, ref. directions
Worker	pymoolab_process_worker.py	Execução em subprocessos isolados

3.1. PymooLab.py — Aplicação Principal

O arquivo principal (**16.461 linhas**) contém toda a aplicação GUI, incluindo:

- **PymooExperimentWindow** — Janela principal com seleção de algoritmos/problemas/métricas, motor de execução, resultados e gráficos
- **AlgorithmSpec, ProblemSpec, MetricSpec, OperatorSpec** — Dataclasses de registro de plugins
- **Plugin discovery** — Varredura automática dos diretórios de plugins
- **Execution engine** — ProcessPoolExecutor + QThread bridge
- **Analysis workspace** — Testes estatísticos, gráficos de convergência, fronteiras de Pareto
- **Export system** — CSV e LaTeX table para resultados de benchmark
- **MCDM** — Seleção de soluções compromisso via multi-critério
- **LLM formulation** — Geração de código via Anthropic

3.2. pymoolab_core/ — Lógica Modular

Subpacote	Módulo	Função
analysis	stat_tests.py	Friedman, Wilcoxon, resumos estatísticos
execution	backend_runtime.py	Detecção de JAX/GPU e status
execution	reproducibility.py	Planos de seed, manifestos SHA-256
llm	formulation.py	Serviço de geração LLM
mcdm	decision.py	Seleção de solução compromisso
registry	backend_selection.py	Resolução backend-aware de operadores
registry	plugin_dirs.py	Scaffolding de diretórios de plugins
registry	rollout.py	Feature gating por estágio

4. Estrutura de Diretórios

```
pymoolab/
■ ■ ■ PymooLab.py           # Aplicação GUI principal
■ ■ ■ styles.py            # Sistema de design centralizado
■ ■ ■ pymoo_metadata.py    # Metadados de algoritmos/problemas pymoo
■ ■ ■ pymoolab_process_worker.py # Worker de subprocesso
■ ■ ■ optimize.py         # Alias legado para pymoo.optimize
■ ■ ■ __init__.py         # Metadados do pacote
■ ■ ■ app.png / pymoolab.png # Imagens da aplicação
■
■ ■ ■ core/                # Classes base de compatibilidade
■   ■ ■ ■ algorithm.py     # Algorithm base (GPU/backend)
■   ■ ■ ■ problem.py      # Re-export pymoo.core.problem
■   ■ ■ ■ population.py   # Re-export pymoo.core.population
■   ■ ■ ■ individual.py   # Re-export pymoo.core.individual
■   ■ ■ ■ mating.py, parameters.py, duplicate.py
■
■ ■ ■ pymoolab_core/      # Lógica modular (Phase 7)
■   ■ ■ ■ analysis/stat_tests.py
■   ■ ■ ■ execution/backend_runtime.py, reproducibility.py
■   ■ ■ ■ llm/formulation.py
■   ■ ■ ■ mcdm/decision.py
■   ■ ■ ■ registry/backend_selection.py, plugin_dirs.py, rollout.py
■
■ ■ ■ algorithms/         # 290 algoritmos locais
■   ■ ■ ■ ssw/ssw.py, ssw_rdpd/ssw_rdpd.py, ...
■
■ ■ ■ problems/           # Problemas de otimização
■   ■ ■ ■ many/ (DTLZ, WFG, MAF, ZCAT, GLSMOP, LSMOP, MAOPP)
■   ■ ■ ■ multi/ (ZDT, CF, UF, MW, RWMOP, DASCMP, ...)
■   ■ ■ ■ single/ (BB0B, CEC2008/2013/2020, ...)
■
■ ■ ■ metrics/            # 25 métricas (HV, IGD, GD, ...)
■   ■ ■ ■ community_metrics.py
■   ■ ■ ■ community_metrics_JAX.py
■
■ ■ ■ operators/          # Operadores genéticos
■   ■ ■ ■ crossover/, mutation/, selection/
■   ■ ■ ■ sampling/, repair/, survival/
■   ■ ■ ■ utility_functions/
■
■ ■ ■ util/               # Utilitários compartilhados
■ ■ ■ tests/              # Testes de benchmark
■ ■ ■ reports/validation/ # Relatórios de validação
```

5. Módulos Principais (Core)

5.1. core/algorithm.py

Classe **Algorithm** que estende `pymoo.core.algorithm.Algorithm` com suporte a backend GPU/JAX:

```
class Algorithm(PymooAlgorithm):
    def __init__(self, *args,
                 use_gpu: bool = False,
                 array_backend: str = "auto",
                 gpu_dtype: str = "float32",
                 **kwargs):
        self.use_gpu_requested = bool(use_gpu)
        self.array_backend_requested = array_backend
        # ... configura backend e delega ao pymoo
        super().__init__(*args, **kwargs)

    def get_array_module(self):
        return array_backend.xp # numpy ou jax.numpy
```

5.2. Spec Dataclasses

O PymooLab registra cada plugin como um dataclass tipado que encapsula seus metadados e factory:

Dataclass	Campos Principais	Propósito
AlgorithmSpec	id, name, source, module, factory, flags	Registro de algoritmo com flags de escopo (single/multi/many)
ProblemSpec	id, name, source, module, factory, default_n_var, default_n_obj	Registro de problema com dimensões padrão
MetricSpec	id, name, source, module, factory	Registro de métrica de qualidade
OperatorSpec	id, name, source, category, module, class_name	Registro de operador genético

6. Sistema de Plugins

O PymooLab utiliza um sistema de **descoberta automática por diretórios**. Na inicialização, a aplicação varre diretórios predefinidos e registra cada módulo Python válido como um plugin.

6.1. Fluxo de Descoberta

```
Startup → ensure_plugin_directories()
        → Scan algorithms/ → Subfolder + __init__.py → AlgorithmSpec
        → Scan problems/   → Arquivo .py → ProblemSpec
        → Scan metrics/    → community_metrics.py → MetricSpec
        → Scan operators/  → Subfolder por categoria → OperatorSpec
        → Scan pymoo.*     → Algoritmos/problemas/operadores nativos
```

6.2. Contratos de Registro

Domínio	Exportação Requerida	Registro
Algorithm	Classe herdando Algorithm ou create_algorithm(config)	AlgorithmSpec com factory
Problem	Classe herdando pymoo.core.problem.Problem	ProblemSpec com factory
Metric	Callable (front, context) → float	MetricSpec com factory
Operator	Classe herdando pymoo.core.{crossover,mutation,...}	OperatorSpec com module + class_name

6.3. ALGORITHM_FLAGS

Cada módulo de algoritmo pode definir um dicionário ALGORITHM_FLAGS que indica quais tipos de problema o algoritmo suporta. Os valores aceitos são: **single**, **multi**, **many**, **neural**.

```
ALGORITHM_FLAGS = {
    "MeuAlgoritmo": {"multi", "many"},
}
```

7. Catálogo Implementado

Snapshot de 03/03/2026. O catálogo inclui plugins locais e nativos do pymoo.

7.1. Algoritmos Locais (290)

ABSAEA, ACMMEA, AdaW, ADSAPSO, AENSGAII, AFSEA, AGEII, AGSEA, AMGPSL, ANSGAIII, APSEA, ARMOEA, AVGSAEA, BCEIBEA, BCEMOEAD, BiCo, BiGE, BLEAQII, BLSAEA, C3M, CAEAD, CAMOEA, CCGDE3, CCMO, cDPEA, CGLP, CLIA, CMaDPPs, CMDEIPCM, CMEGL, CMME, CMMO, CMOBR, CMOCSSO, CMODEFTR, CMODRL, CMOECD, CMOEAD, CMOEAMS, CMOEAMSG, CMOEBOB, CMOEMT, CMOES, CMOQLMT, CMOSMA, CNSDEDVC, CoMMEA, CPSMOEA, CSEA, CSEMT, CTSEA, DAEA, DBEMTO, DCNSGAIII, DEAGNG, DGEA, DirHVEI, DISK, DISK_2025, DISKplus, DKCA, DMMOEA, DMOEAEc, dMOPSO, DNNSGAII, DPCPRA, DPPPS, DPVAPS, DRLOSEMCMO, DRLSAEA, DSPCMDE, DSSEA, DVCEA, DWU, EAGMOEAD, EDNARMOEA, EFRRR, EIMEGO, EM_SAEA, EMCMMMS, EMCMO, EMMOEA, eMOEA, EMOSKT, EMyOC, ENSMOEAD, ESBCEO, FDV, FLEA, FRCGM, GCNMOEA, GDE3, GFMMOEA, GLMO, gNSGAII, GPSOM, GrEA, GWASFGEA, HEA, HeEMOEA, HHCMMEA, hpaEA, HREA, HypE, IBEA, ICMA, IDBEA, IMCMOEAD, IMMOEA, IMMOEAD, IMTCMO, IMTCMO_BS, ISIBEA, Izui, KLEA, KLNSGAII, KnEA, KRVEA, KTA2, KTS, LCMEA, LCSA, LDSAF, LERD, LMEA, LMOCSSO, LMOEADS, LMPFE, LRMOEA, LSMOF, MaOEACSS, MaOEADDFC, MaOEAGID, MaOEAIT, MaOEARD, MCCMO, MCEAD, MFFS, MFOSPEA2, MGCEA, MGSAEA, MMEAPSL, MMEAWI, MMOPSO, MO_Ring_PSO_SCD, MOBCA, MOCeII, MOCGDE, MOCMA, MOEACKF, MOEAD2WA, MOEADAWA, MOEADCMA, MOEADCMT, MOEADD, MOEADDAE, MOEADDCWV, MOEADDE, MOEADDQN, MOEADDRA, MOEADDU, MOEADDYTS, MOEADDEGO, MOEADFRRMAB, MOEADM2M, MOEADMRDL, MOEADPaS, MOEADPFE, MOEADSTM, MOEADUR, MOEADURAW, MOEADVA, MOEADVOV, MOEAIGDNS, MOEANZD, MOEAPC, MOEAPSL, MOEARE, MOEGS, MOL2SMEA, MOMBIII, MOMFEA, MOMFEAII, MOMFEASADE, MONAS, MOPSO, MOSD, MPAES, MPMMEA, MPSOD, MSCEA, MSCMO, MSEA, MSKEA, MSOPSII, MTCMO, MTDEMKA, MTEADDN, MTS, MultiObjectiveEGO, MyODEMR, NBLEA, NMPSO, NNDREAMO, NNIA, NRVMOEA, NSBiDiCo, NSGAIIARSBX, NSGAIIconflict, NSGAIIDTI, NSGAIIIEHVI, NSGAIIISDR, NSLS, NUCEA, onebyoneEA, OSPNSDE, ParEGO, PBNSGAIII, PBRVEA, PCSAEA, PEA, PEApIus, PeEA, PESAI, PICEAg, PIEA, PIMD, PMMOEA, POCEA, PPS, PRDH, PREA, REMO, RGA_M1_2, RGA_M2_2, RMMEDA, RMOEADVA, RPDNSGAII, RPEA, RSEA, RVEAa, RVEAiGNG, S3CMAES, SAMOEA_TL2M, SCDAS, SCEA, SECSO, SFADE, SGEA, SGECF, SIBEA, SIBEAkEMOSS, SLMEA, SMEA, SMOA, SMPSO, SMSEGO, SNSGAII, SparseEA, SparseEA2, SPEA2SDE, SPEAR, SRA, SSCEA, SSDE, SSW, SSW_RDPA, SVRNSGAII, tDEA, tDEACPBI, TEA, TELSO, TiGE2, ToP, TPCMaO, TriMOEATAR, TSNSGAII, TSSparseEA, TSTI, Two_Arch2, URCMO, VaEA, WASFGA, WOF, WVMOEAP

Além dos algoritmos locais, o PymooLab inclui todos os algoritmos nativos do pymoo: NSGA-II, NSGA-III, U-NSGA-III, RVEA, MOEA/D, AGE-MOEA, AGE-MOEA-II, C-TAEA, SMS-EMOA, SPEA2, entre outros.

7.2. Métricas (25)

CPF, DeltaP, DM, Feasible_rate, GD, HV, IGD, IGDp, IGDx, Lower_level_Min_value, Mean_HV, Mean_IGD, Min_value, PD, Spacing, Spread, Task1_HV, Task1_IGD, Task1_Min_value, Task2_HV, Task2_IGD, Task2_Min_value, Upper_level_Min_value, Worst_HV, Worst_IGD

7.3. Operadores (38)

AgeBasedTournamentSelection, BFM, BinaryRandomSampling, BinomialCrossover, BitflipMutation, BX, ChoiceRandomMutation, DEX, EdgeRecombinationCrossover, ERX, ExponentialCrossover, FloatRandomSampling, GaussianMutation, GM, HalfUniformCrossover, HUX, IntegerRandomSampling,

InversionMutation, LatinHypercubeSampling, LHS, NoCrossover, NoMutation, OrderCrossover, ParentCentricCrossover, PCX, PermutationRandomSampling, PM, PointCrossover, PolynomialMutation, RandomSelection, SBX, SimulatedBinaryCrossover, SinglePointCrossover, SPX, TournamentSelection, TwoPointCrossover, UniformCrossover, UX

7.4. Famílias de Problemas (41)

BBOB, BT, CF, DASCOP, DOC, DSMOP, DTLZ, FCP, FDA, GLSMOP, IMMOEA, IMOP, LIRCMOP, LRMOP, LSCM, LSMMOP, LSMOP, MAF, MAOPP, MMF, MMMOP, MOEADDE, MOEADM2M, MULTITASKING_MOPS, MULTITASKING_SOPS, MW, REALWORLD_MOPS, RMMEDA, RWMOP, SDC, SIMPLE_SOPS, SMD, SMMOP, SMOP, TP, UF, VNT, WFG, ZCAT, ZDT, ZXH_CF

8. Criando Novos Problemas

8.1. Localização do Arquivo

Crie um arquivo .py no subdiretório correspondente ao número de objetivos:

Diretório	Tipo de Problema	Exemplo
problems/single/	Single-objective (1 obj)	Funções CEC, BBOB
problems/multi/	Multi-objective (2–3 obj)	ZDT, CF, UF
problems/many/	Many-objective (≥ 4 obj)	DTLZ, WFG, MAF

8.2. Padrão de Implementação

Todo problema deve herdar de `pymoo.core.problem.Problem` e implementar o método `_evaluate()`:

```
from pymoo.core.problem import Problem
import numpy as np

class MeuProblema(Problem):
    def __init__(self, n_var: int = 10, **kwargs):
        super().__init__(
            n_var=int(n_var),
            n_obj=2,          # Número de objetivos
            n_ieq_constr=0,   # Restrições de desigualdade
            xl=0.0,          # Limite inferior
            xu=1.0,          # Limite superior
            vtype=float,
            **kwargs,
        )

    def _evaluate(self, x, out, *args, **kwargs):
        x = np.asarray(x, dtype=float)
        f1 = x[:, 0]
        g = 1.0 + 9.0 * np.mean(x[:, 1:], axis=1)
        f2 = g * (1.0 - np.sqrt(f1 / g))
        out["F"] = np.column_stack([f1, f2])

    def _calc_pareto_front(self, n_pareto_points: int = 200):
        # Fronteira de Pareto analítica (opcional, melhora métricas)
        x = np.linspace(0.0, 1.0, n_pareto_points)
        return np.column_stack([x, 1.0 - np.sqrt(x)])

__all__ = ["MeuProblema"]
```

8.3. Suporte JAX (Opcional)

Para acelerar a avaliação com JAX, crie uma variante com `_USE_JAX = True`:

```
class MeuProblema_JAX(MeuProblema):
    _USE_JAX = True

__all__ = ["MeuProblema", "MeuProblema_JAX"]
```

9. Criando Novos Algoritmos

9.1. Estrutura de Diretório

```
algorithms/  
■ ■ ■ meu_algoritmo/  
    ■ ■ ■ __init__.py      # Exporta a classe ou factory  
    ■ ■ ■ meu_algoritmo.py # Implementação
```

9.2. Implementação Completa

```
# algorithms/meu_algoritmo/meu_algoritmo.py  
  
from __future__ import annotations  
import numpy as np  
from core.algorithm import Algorithm  
from core.population import Population  
  
# Flags de escopo (obrigatório para filtragem na GUI)  
ALGORITHM_FLAGS = {  
    "MeuAlgoritmo": {"multi", "many"},  
}  
  
class MeuAlgoritmo(Algorithm):  
    """Algoritmo customizado para otimização multiobjetivo."""  
  
    def __init__(self, pop_size: int = 100, **kwargs):  
        super().__init__(**kwargs)  
        self.pop_size = pop_size  
  
    def _setup(self, problem, **kwargs):  
        """Armazena metadados do problema."""  
        self.n_var = problem.n_var  
        self.n_obj = problem.n_obj  
        self.xl = problem.xl  
        self.xu = problem.xu  
  
    def _initialize_infill(self):  
        """Gera população inicial aleatória."""  
        X = np.random.uniform(self.xl, self.xu,  
                               (self.pop_size, self.n_var))  
        return Population.new("X", X)  
  
    def _infill(self):  
        """Gera novas soluções a cada iteração."""  
        X = np.random.uniform(self.xl, self.xu,  
                               (self.pop_size, self.n_var))  
        return Population.new("X", X)  
  
    def _advance(self, infills=None, **kwargs):  
        """Atualiza população com soluções avaliadas."""  
        self.pop = infills
```

9.3. Arquivo __init__.py

```
# algorithms/meu_algoritmo/__init__.py  
from .meu_algoritmo import MeuAlgoritmo  
__all__ = ["MeuAlgoritmo"]
```

9.4. Factory Function (Alternativa)

Em vez de exportar a classe diretamente, você pode exportar uma função `create_algorithm(config: dict)`:

```
def create_algorithm(config: dict):  
    pop_size = config.get("pop_size", 100)  
    return MeuAlgoritmo(pop_size=pop_size)
```

10. Criando Novas Métricas

10.1. Contrato da Métrica

Uma métrica é uma **callable** que recebe a fronteira obtida e um contexto, retornando um **float**:

```
def minha_metrica(front: np.ndarray, context: dict) -> float:
    # front: (N, M) – N soluções, M objetivos
    # context: dicionário com 'problem', 'reference_front', etc.
    return float(valor_calculado)
```

10.2. Contexto Disponível

Chave	Tipo	Descrição
problem	Problem	Instância do problema sendo avaliado
reference_front	np.ndarray None	Fronteira de Pareto verdadeira
reference_set	np.ndarray None	Conjunto de referência no espaço X
n_obj	int	Número de objetivos
pop_X	np.ndarray None	Variáveis de decisão da população
pop_G	np.ndarray None	Violações de restrição
feasible_mask	np.ndarray None	Máscara de factibilidade

10.3. Exemplo Completo

```
# metrics/minha_metrica.py (ou adicionar em community_metrics.py)
import numpy as np
from typing import Any

def _metric_MinhaDist(front: np.ndarray,
                      context: dict[str, Any]) -> float:
    """Distância média ao ponto ideal."""
    pop_obj = np.asarray(front, dtype=float)
    if pop_obj.ndim == 1:
        pop_obj = pop_obj[np.newaxis, :]
    ideal = np.min(pop_obj, axis=0)
    distances = np.sqrt(np.sum((pop_obj - ideal) ** 2, axis=1))
    return float(np.mean(distances))

# Registro via dicionário METRICS:
METRICS = {
    "MinhaDist": _metric_MinhaDist,
}
```

11. Criando Novos Operadores

11.1. Categorias de Operadores

Categoria	Classe Base pymoo	Diretório
Crossover	pymoo.core.crossover.Crossover	operators/crossover/
Mutation	pymoo.core.mutation.Mutation	operators/mutation/
Selection	pymoo.core.selection.Selection	operators/selection/
Sampling	pymoo.core.sampling.Sampling	operators/sampling/

11.2. Exemplo: Crossover Customizado

```
# operators/crossover/meu_crossover.py
from __future__ import annotations
import numpy as np
from pymoo.core.crossover import Crossover

class MeuCrossover(Crossover):
    """Crossover aritmético simples."""

    def __init__(self, alpha: float = 0.5, **kwargs):
        super().__init__(n_parents=2, n_offsprings=2, **kwargs)
        self.alpha = alpha

    def _do(self, problem, X, **kwargs):
        n_matings, n_parents, n_var = X.shape
        Y = np.full((n_matings, self.n_offsprings, n_var), np.nan)
        for i in range(n_matings):
            p1, p2 = X[i, 0], X[i, 1]
            Y[i, 0] = self.alpha * p1 + (1 - self.alpha) * p2
            Y[i, 1] = (1 - self.alpha) * p1 + self.alpha * p2
        return Y
```

11.3. Exemplo: Mutation Customizada

```
# operators/mutation/minha_mutacao.py
import numpy as np
from pymoo.core.mutation import Mutation

class MinhaMutacao(Mutation):
    """Mutaç o gaussiana customizada."""

    def __init__(self, sigma: float = 0.1, **kwargs):
        super().__init__(**kwargs)
        self.sigma = sigma

    def _do(self, problem, X, **kwargs):
        X = np.asarray(X, dtype=float)
        noise = np.random.normal(0, self.sigma, X.shape)
        Y = X + noise
        Y = np.clip(Y, problem.xl, problem.xu)
        return Y
```

12. Execução e Uso do PymooLab

12.1. Iniciando a Aplicação

```
python PymooLab.py
```

A aplicação inicia em modo maximizado com tema claro (light_blue.xml via qt-material). Todos os diretórios de plugins são criados automaticamente se não existirem.

12.2. Fluxo de Experimentação

- 1. Selecionar Algoritmo(s):** Use os filtros por tipo (single/multi/many) e encoding (real/binary/integer/permutation) para encontrar algoritmos.
- 2. Selecionar Problema(s):** Filtre por família, tipo de objetivo e dificuldade (constrained/dynamic).
- 3. Selecionar Métrica(s):** HV, IGD, GD, Spacing, etc.
- 4. Configurar Execução:** Max FE, número de runs, modo de seed (random/fixed/sequence), backend (CPU/JAX).
- 5. Executar:** O motor de execução usa ProcessPoolExecutor para paralelizar trials em subprocessos isolados.
- 6. Analisar Resultados:** Gráficos de convergência, fronteiras de Pareto, testes estatísticos (Friedman, Wilcoxon), exportação CSV/LaTeX.
- 7. MCDM (Opcional):** Selecionar solução compromisso das fronteiras obtidas.

12.3. Configurações de Reprodutibilidade

Modo de Seed	Comportamento	Determinístico?
random	Seed aleatório por trial	Não
fixed	Mesmo seed para todos os trials	Sim
sequence	Sequência explícita de seeds	Sim

Apêndice: Referências

- [1] Santos, T. F., de Souza, G., Carneiro, L. G. O. & Xavier, S. (2025). PymooLab: An Open-Source Visual Analytics Framework for Multi-Objective Optimization using LLM-Based Code Generation and MCDM. arXiv:2603.01345. DOI: 10.48550/arXiv.2603.01345.
- [2] Blank, J. & Deb, K. (2020). pymoo: Multi-Objective Optimization in Python. IEEE Access, 8, 89497-89509.
- [3] Deb, K. et al. (2002). A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II. IEEE TEC, 6(2), 182-197.
- [4] Zitzler, E. & Thiele, L. (1999). Multiobjective Evolutionary Algorithms: A Comparative Case Study. IEEE TEC, 3(4), 257-271.
- [5] Huband, S. et al. (2006). A Review of Multiobjective Test Problems and a Scalable Test Problem Toolkit. IEEE TEC, 10(5), 477-506.
- [6] Schäffler, S. et al. (2002). Stochastic Method for the Solution of Unconstrained Vector Optimization Problems. JOTA, 114(1), 209-222.
- [7] While, L. et al. (2012). A Fast Way of Calculating Exact Hypervolumes. IEEE TEC, 16(1), 86-95.

METISBr — Multi-Objective and Many-Objective Optimization Research Group
UFOP — Brasil

Documento gerado em 03/03/2026 às 18:23